ITMO UNIVERSITY

Встроенные системы

Мастер-класс 3

«Программирование микроконтроллеров с операционной системой (FreeRTOS)»

ITMOre than a UNIVERSITY

ITMO UNIVERSITY

План занятия

Теоретическая часть

- FreeRTOS
- Задачи
- Алгоритмы планирования
- Межпроцессное взаимодействие
- Потенциальные проблемы

Практическая часть:

- Работа с FreeRTOS в Stm32CubeIDE
- Обсуждение задания

ITMO more than a UNIVERSITY

FreeRTOS

ITMO UNIVERSITY

FreeRTOS — это ядро реального времени (или планировщик реального времени), на основе которого можно создавать встроенные приложения, отвечающие жестким требованиям реального времени.

Мягкое реальное время: нарушение крайнего срока не сделает систему бесполезной. Например, слишком медленная реакция на нажатия клавиш может сделать систему раздражающе неотзывчивой, не делая ее фактически непригодной для использования.

Жесткое реальное время: нарушение крайнего срока приведет к полному отказу системы. Например, подушка безопасности водителя может принести больше вреда, чем пользы, если она слишком медленно отреагирует на сигналы датчика столкновения.

ITMO more than a UNIVERSITY

Преимущества FreeRTOS

ITMO UNIVERSITY

- Абстрагирование информации о времени
- Поддерживаемость/Расширяемость
- Модульность
- Командная разработка
- Упрощенное тестирование
- Повторное использование кода
- Гибкая обработка прерываний

ITMO more than a UNIVERSITY

Возможности FreeRTOS

ITMO UNIVERSITY

- Упреждающая или кооперативная работа
- Очень гибкое назначение приоритета задач
- Гибкий, быстрый и легкий механизм уведомления о задачах
- Очереди
- Двоичные семафоры
- Счетные семафоры
- Мьютексы
- Рекурсивные мьютексы
- Программные таймеры
- Группы событий
- Функции перехвата тиков
- Функции перехвата простоя
- Проверка переполнения стека
- Запись трассировки
- Сбор статистики времени выполнения задачи
- Необязательное коммерческое лицензирование и поддержка
- Полная модель вложенности прерываний (для некоторых архитектур)
- Программное управление стеком прерываний при необходимости (это может помочь сэкономить ОЗУ)

ITMO more than a UNIVERSITY

Задачи во FreeRTOS

ITMO UNIVERSITY

- Приложения в FreeRTOS обычно структурированы как набор автономных задач
- Каждая задача выполняется в своем собственном контексте (например, стеке) без случайной зависимости от других задач
- Планировщик запускает, останавливает и меняет местами задачи

ITMO more than a UNIVERSITY

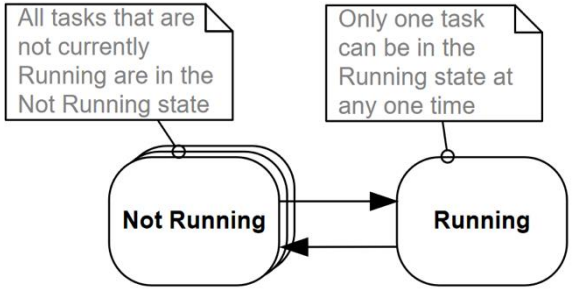
Задачи во FreeRTOS

ITMO UNIVERSITY

- Задача может находиться в одном из двух состояний: «Выполняется» и «Не выполняется».

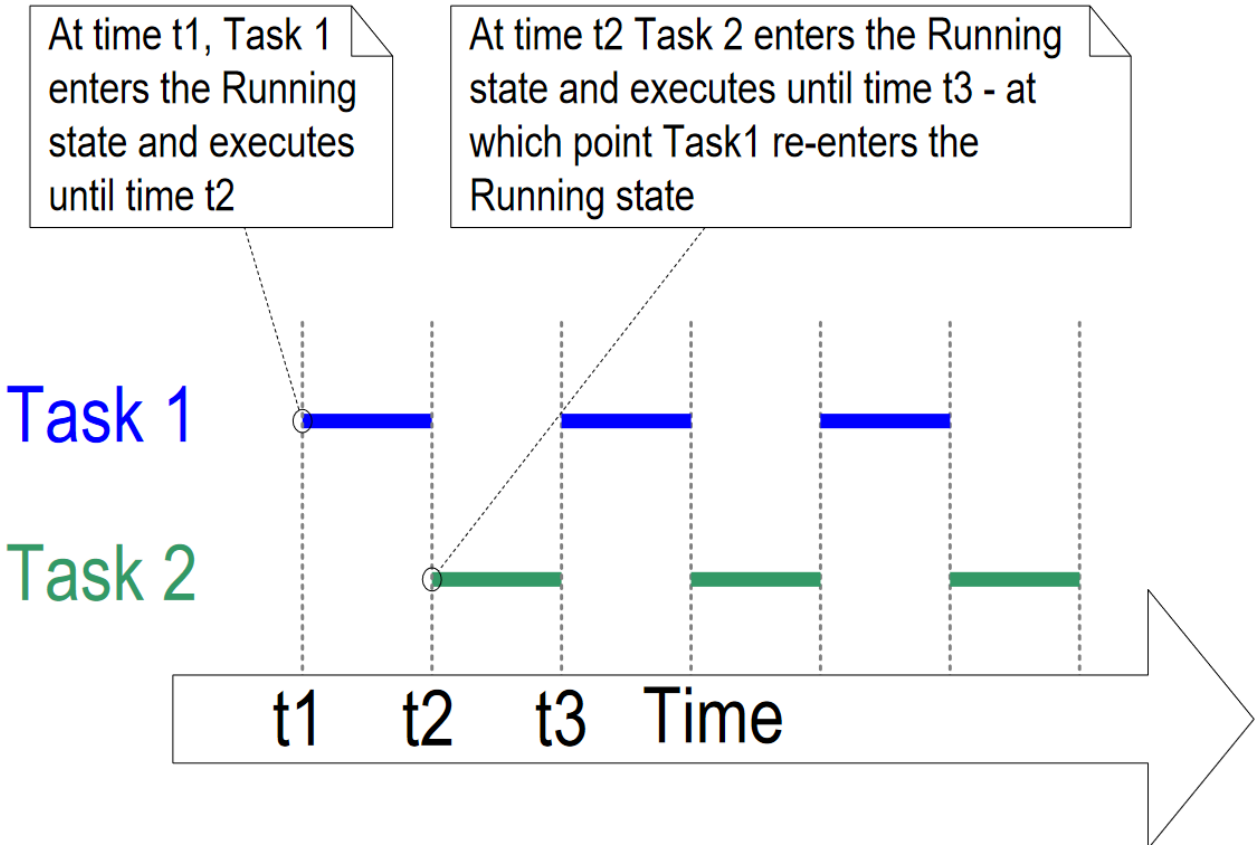
- Когда задача находится в состоянии «Выполняется», процессор выполняет код задачи. Когда задача находится в состоянии «Не выполняется», она находится в состоянии покоя, ее статус сохранен и готов к возобновлению выполнения в следующий раз, когда планировщик решит, что она должна перейти в состояние «Выполняется».

- Когда задача возобновляет выполнение, она делает это с инструкции, которую она собиралась выполнить до того, как в последний раз вышла из состояния «Выполняется».



ITMO More than a UNIVERSITY

Задачи во FreeRTOS. Пример



Задачи во FreeRTOS. Приоритеты

ITMO UNIVERSITY

- Параметр uxPriority функции API xTaskCreate() назначает начальный приоритет создаваемой задаче. Приоритет можно изменить после запуска планировщика с помощью функции API vTaskPrioritySet().

- Максимальное количество доступных приоритетов задается константой конфигурации времени компиляции configMAX_PRIORITIES, определяемой приложением, в FreeRTOSConfig.h. Низкие числовые значения приоритета обозначают задачи с низким приоритетом, а приоритет 0 является самым низким возможным приоритетом.

- Диапазон доступных приоритетов составляет от 0 до (configMAX_PRIORITIES – 1). Любое количество задач может иметь один и тот же приоритет, что обеспечивает максимальную гибкость проектирования.

ITMO more than a UNIVERSITY

Задачи во FreeRTOS. Планирование задач

ITMO UNIVERSITY

- Задача, которая фактически выполняется (используя время обработки), находится в состоянии выполнения. На одноядерном процессоре в любой момент времени может быть только одна задача в состоянии выполнения.

- Задачи, которые фактически не выполняются, но не находятся ни в состоянии блокировки, ни в состоянии приостановки, находятся в состоянии готовности. Задачи, которые находятся в состоянии готовности, доступны для выбора планировщиком в качестве задачи для перехода в состояние выполнения.

- Планировщик всегда будет выбирать задачу с наивысшим приоритетом в состоянии готовности для перехода в состояние выполнения.

ITMO more than a UNIVERSITY

Задачи во FreeRTOS. Планирование задач

ITMO UNIVERSITY

- Задачи могут ожидать в состоянии блокировки события и автоматически возвращаются в состояние готовности, когда событие происходит. Временные события происходят в определенное время, например, когда истекает время блокировки, и обычно используются для реализации периодического или поведения с таймаутами.

- События синхронизации происходят, когда задача или процедура обслуживания прерывания отправляет информацию с помощью уведомления о задаче, очереди, группы событий или одного из многих типов семафора. Обычно они используются для сигнализации об асинхронной активности, например, о поступлении данных на периферийное устройство.

Алгоритмы планирования

ITMO UNIVERSITY

- Алгоритм планирования — это программная процедура, которая решает, какую задачу из состояния готовности перевести в состояние выполнения.

- Алгоритм можно изменить с помощью констант конфигурации configUSE_PREEMPTION и configUSE_TIME_SLICING. Обе константы определены в FreeRTOSConfig.h.

- **Приоритетное вытесняющее планирование с квантованием по времени**

configUSE_PREEMPTION 1 && configUSE_TIME_SLICING 1

- **Приоритетное вытесняющее планирование без квантования по времени**

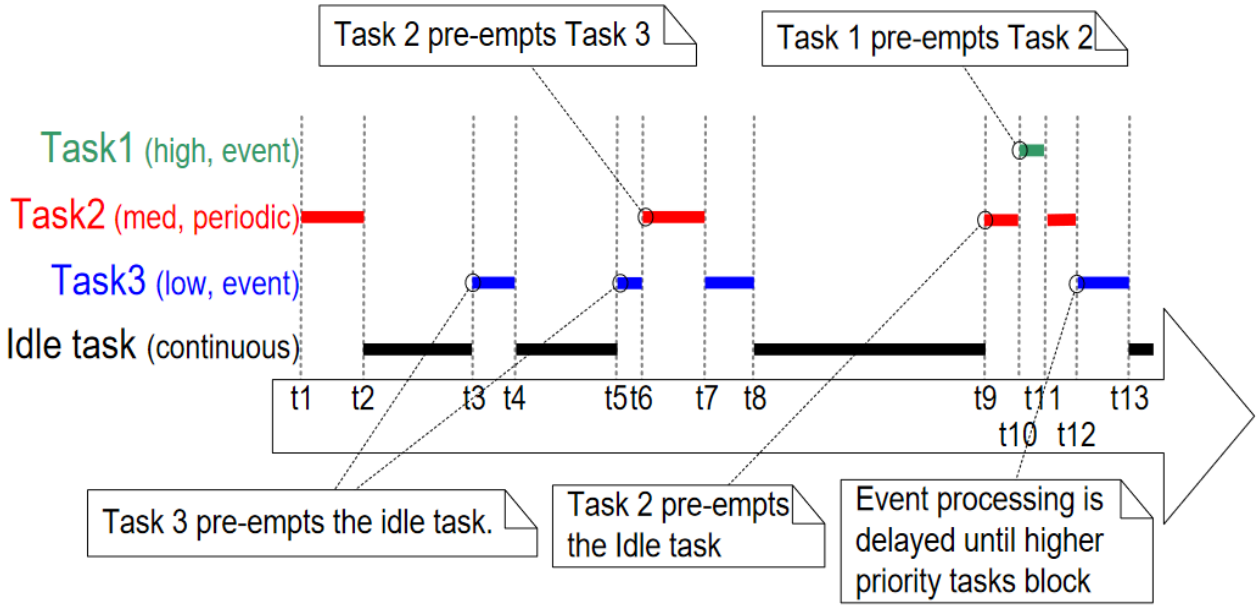
configUSE_PREEMPTION 1 && configUSE_TIME_SLICING 0

- **Кооперативное (совместное) планирование**

configUSE_PREEMPTION 0 && configUSE_TIME_SLICING 0/1

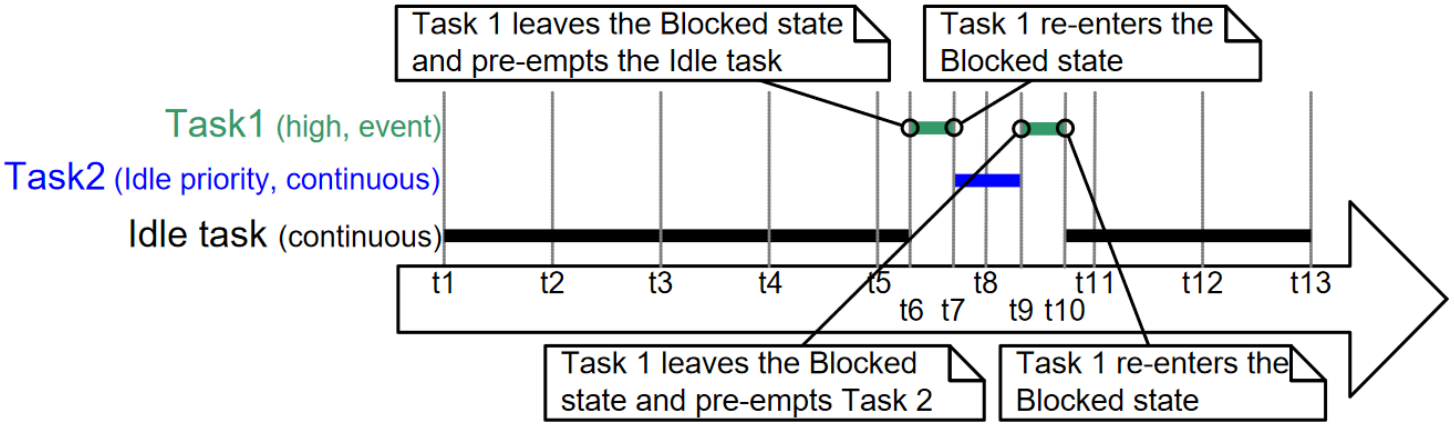
ITMO More than a UNIVERSITY

Алгоритмы планирования



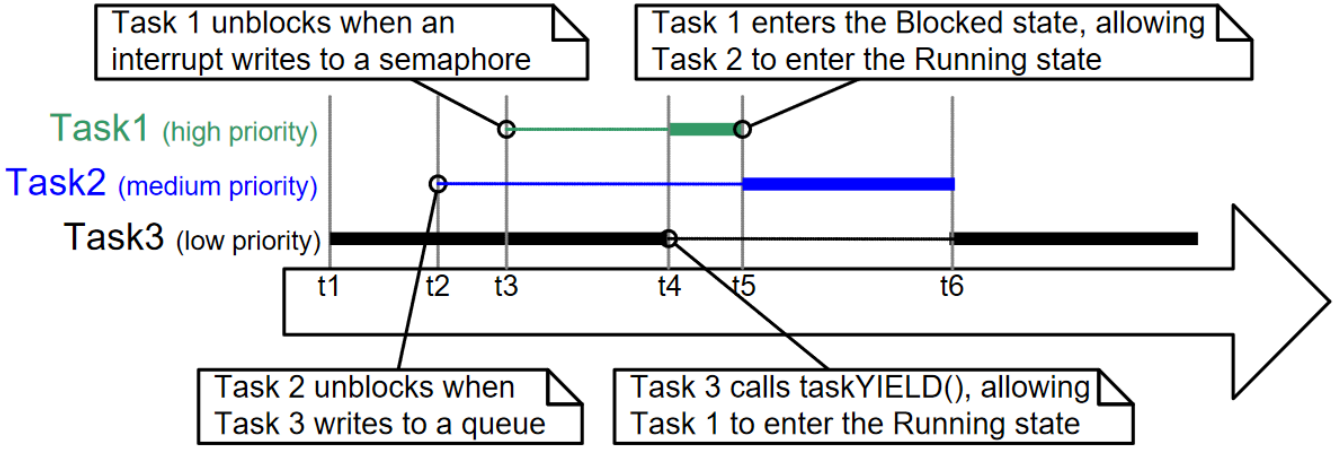
Приоритетное вытесняющее планирование с квантованием по времени

Алгоритмы планирования



Приоритетное вытесняющее планирование без квантования по времени

Алгоритмы планирования



Кооперативное планирование

Межпроцессное взаимодействие

ITMO UNIVERSITY

- Межпроцессное взаимодействие (IPC) — это механизм, позволяющий процессам взаимодействовать друг с другом и синхронизировать свои действия.

- Когда процессы должны иметь возможность общаться с другими процессами, должен быть хорошо структурированный способ общения без каких-либо прерываний, вот где вступает в дело межпроцессное общение.

- Несколько типов механизмов, используемых для IPC:
  - ✓ Синхронизация
  - ✓ Совместное использование памяти
  - ✓ Передача сообщений

ITMO more than a UNIVERSITY

Межпроцессное взаимодействие

ITMO UNIVERSITY

- Очереди (Queues)
- Бинарные семафоры (Binary semaphores)
- Счетные семафоры (Counting semaphores)
- Мьютексы (Mutexes)
- Рекурсивные мьютексы (Recursive Mutexes)

ITMO more than a UNIVERSITY

Очереди в FreeRTOS

ITMO UNIVERSITY

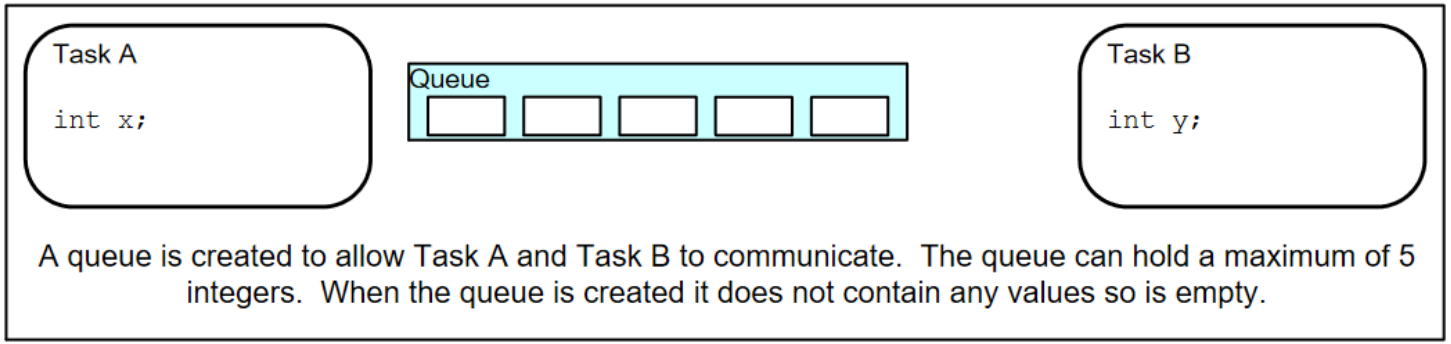
- Очередь может содержать конечное число элементов данных фиксированного размера. Максимальное число элементов, которое может содержать очередь, называется ее «длиной». Длина и размер каждого элемента данных устанавливаются при создании очереди.

- Очереди обычно используются как буферы FIFO (First In First Out), где данные записываются в конец (хвост) очереди и удаляются из начала (головы) очереди.

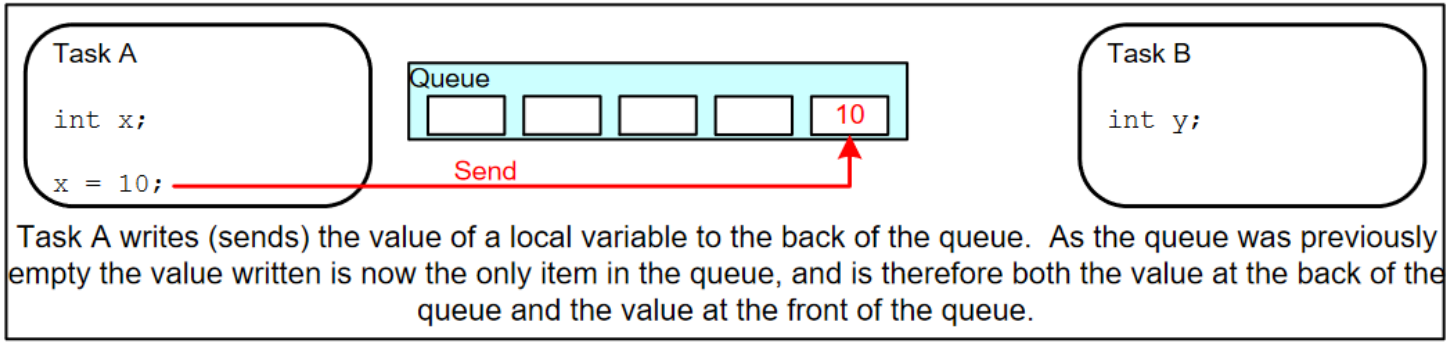
- Также можно записывать в начало очереди и перезаписывать данные, которые уже находятся в начале очереди.

ITMO more than a UNIVERSITY

Очереди в FreeRTOS. Пример

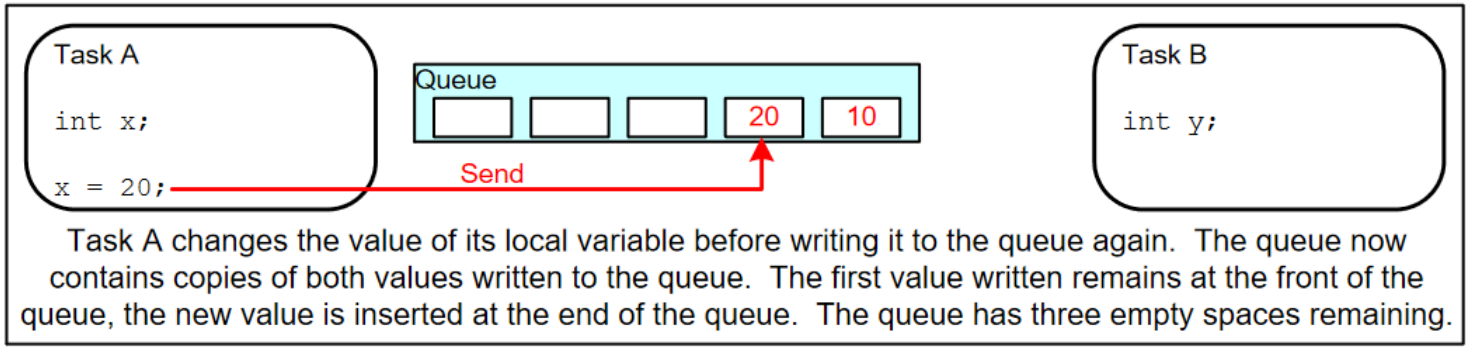


A queue is created to allow Task A and Task B to communicate. The queue can hold a maximum of 5 integers. When the queue is created it does not contain any values so is empty.

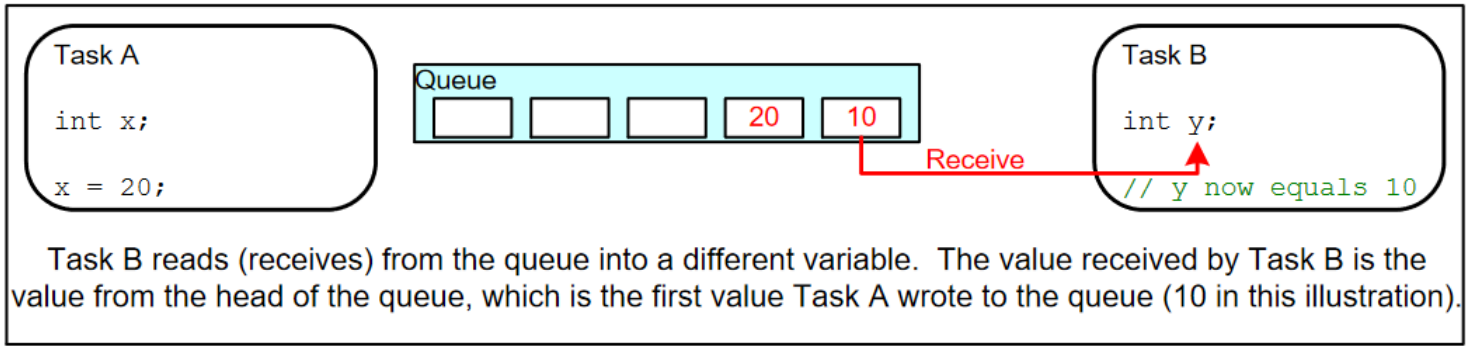


Task A writes (sends) the value of a local variable to the back of the queue. As the queue was previously empty the value written is now the only item in the queue, and is therefore both the value at the back of the queue and the value at the front of the queue.

Очереди в FreeRTOS. Пример

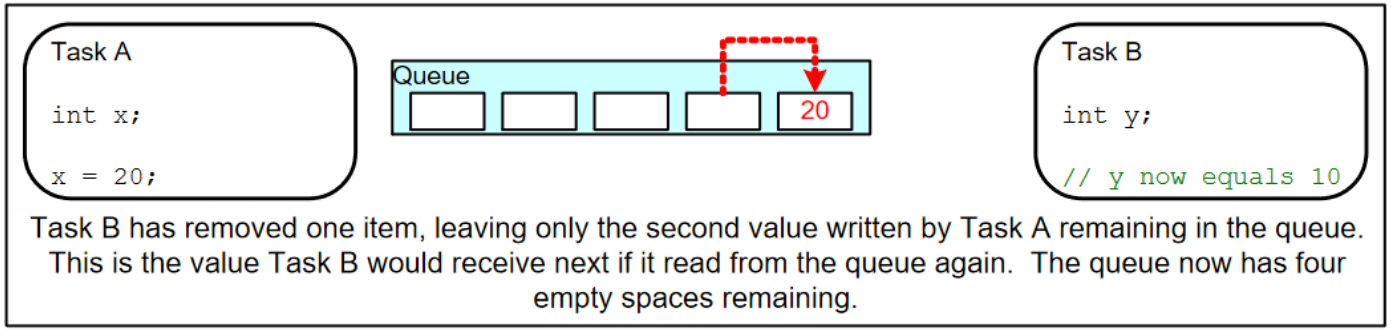


Task A changes the value of its local variable before writing it to the queue again. The queue now contains copies of both values written to the queue. The first value written remains at the front of the queue, the new value is inserted at the end of the queue. The queue has three empty spaces remaining.



Task B reads (receives) from the queue into a different variable. The value received by Task B is the value from the head of the queue, which is the first value Task A wrote to the queue (10 in this illustration).

Очереди в FreeRTOS. Пример



Task B has removed one item, leaving only the second value written by Task A remaining in the queue. This is the value Task B would receive next if it read from the queue again. The queue now has four empty spaces remaining.

Реализация очереди FreeRTOS

ITMO UNIVERSITY

Существует два способа реализации поведения очереди:

1. Очередь путем копирования
Очередь путем копирования означает, что данные, отправленные в очередь, копируются в очередь байт за байтом.

2. Очередь по ссылке
Очередь по ссылке означает, что очередь хранит только указатели на данные, отправленные в очередь, а не сами данные.

ITMO more than a UNIVERSITY

Доступ к очереди

ITMO UNIVERSITY

- Очереди — это объекты, к которым может получить доступ любая задача или ISR, знающая об их существовании.
- Любое количество задач может писать в одну и ту же очередь, и любое количество задач может читать из одной и той же очереди.
- На практике очень часто бывает так, что очередь имеет несколько писателей, но гораздо реже бывает так, что очередь имеет несколько читателей.

Доступ к очереди. Блокировка

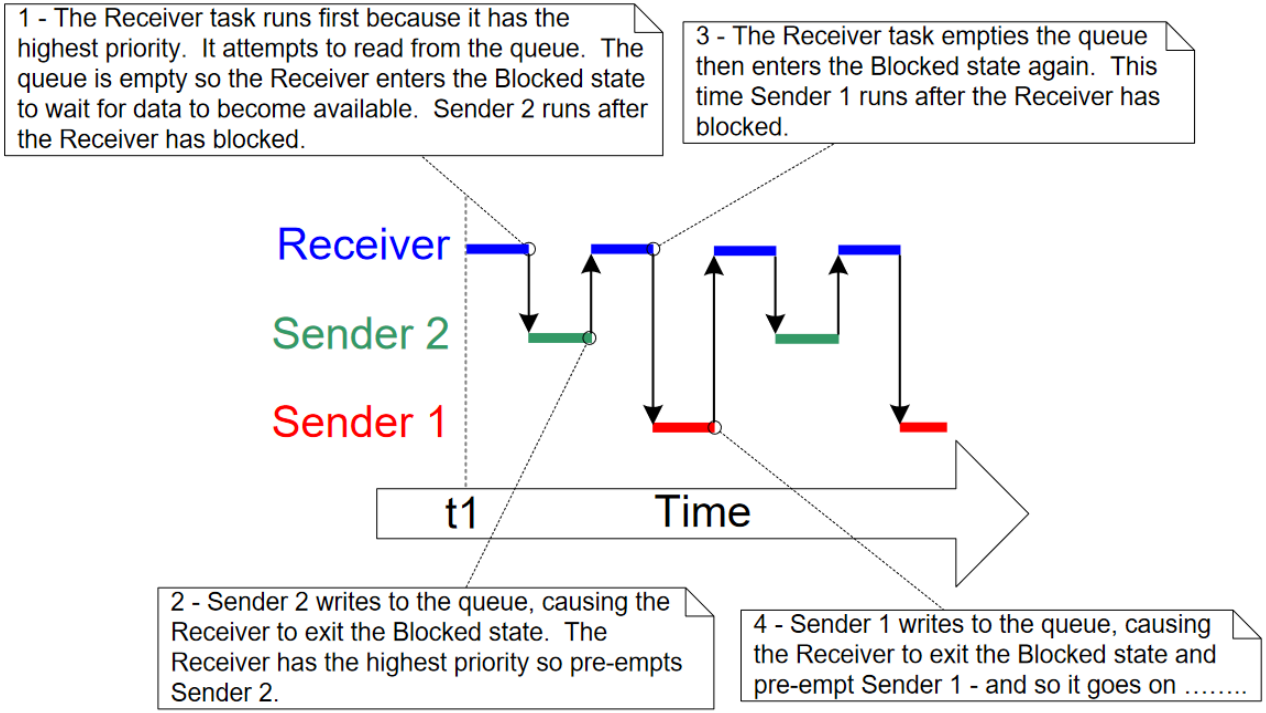
ITMO UNIVERSITY

- Когда задача пытается прочитать из очереди, она может опционально указать время «блокировки». Это время, в течение которого задача будет находиться в заблокированном состоянии, ожидая, пока данные станут доступны из очереди, если очередь уже пуста. Задача, которая находится в заблокированном состоянии, ожидая, пока данные станут доступны из очереди, автоматически переходит в состояние готовности, когда другая задача или прерывание помещает данные в очередь. Задача также будет автоматически переведена из заблокированного состояния в состояние готовности, если указанное время блокировки истекает до того, как данные станут доступны.

- Так же, как при чтении из очереди, задача может опционально указать время блокировки при записи в очередь. В этом случае время блокировки — это максимальное время, в течение которого задача должна находиться в заблокированном состоянии, ожидая, пока в очереди не станет доступно место, если очередь уже заполнена.

ITMOre than a UNIVERSITY

Очередь. Пример



Указатели и очередь

ITMO UNIVERSITY

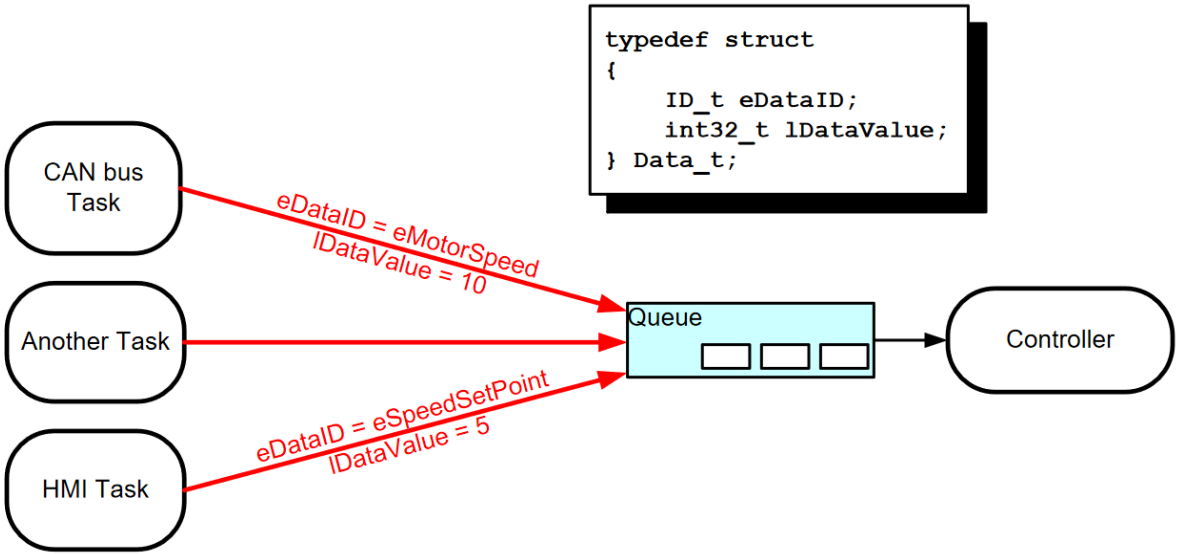
Если размер данных, хранящихся в очереди, большой, то предпочтительнее использовать очередь для передачи указателей на данные, а не копировать сами данные в очередь и из нее побайтно.

Передача указателей более эффективна как по времени обработки, так и по объему оперативной памяти, необходимой для создания очереди. Однако при размещении указателей в очереди необходимо проявлять особую осторожность.

Данные из нескольких источников

ITMO UNIVERSITY

В проектах FreeRTOS часто бывает так, что задача получает данные из более чем одного источника. Получающая задача должна знать, откуда пришли данные, чтобы определить, как их следует обрабатывать. Простым решением при проектировании является использование одной очереди для передачи структур, в которых как значение данных, так и источник данных содержатся в полях структуры.



ITMO more than a UNIVERSITY

Отложенная обработка прерываний

ITMO UNIVERSITY

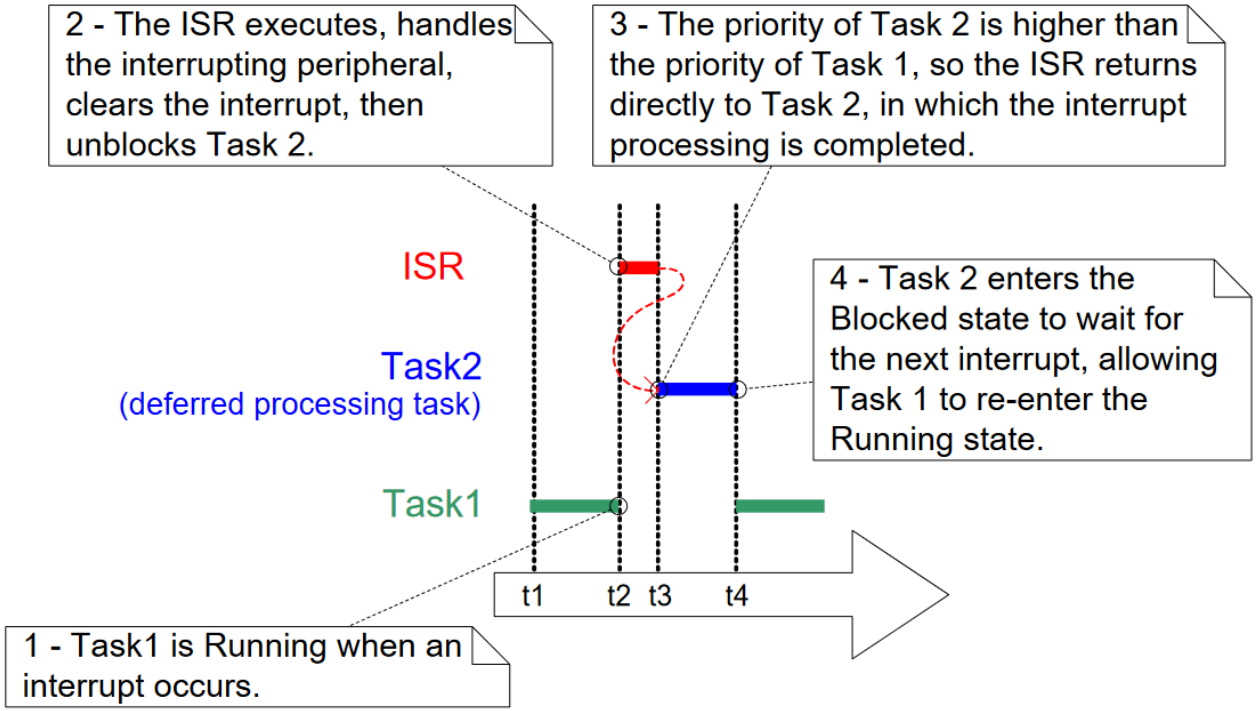
Обычно считается лучшей практикой делать ISR как можно короче. Причины этого включают:

- Даже если задачам был назначен очень высокий приоритет, они будут выполняться только в том случае, если никакие прерывания не обслуживаются оборудованием.
- ISR могут нарушать как время запуска, так и время выполнения задачи.
- В зависимости от архитектуры, на которой работает FreeRTOS, может быть невозможно принять какие-либо новые прерывания или, по крайней мере, подмножество новых прерываний во время выполнения ISR.
- Автор приложения должен учитывать последствия и защищаться от таких ресурсов, как переменные, периферийные устройства и буферы памяти, к которым одновременно обращаются задача и ISR.

Любая другая обработка, требуемая прерыванием, часто может выполняться в задаче, позволяя процедуре обработки прерывания завершиться так быстро, как это практически возможно.

ITMO More than a UNIVERSITY

Отложенная обработка прерываний



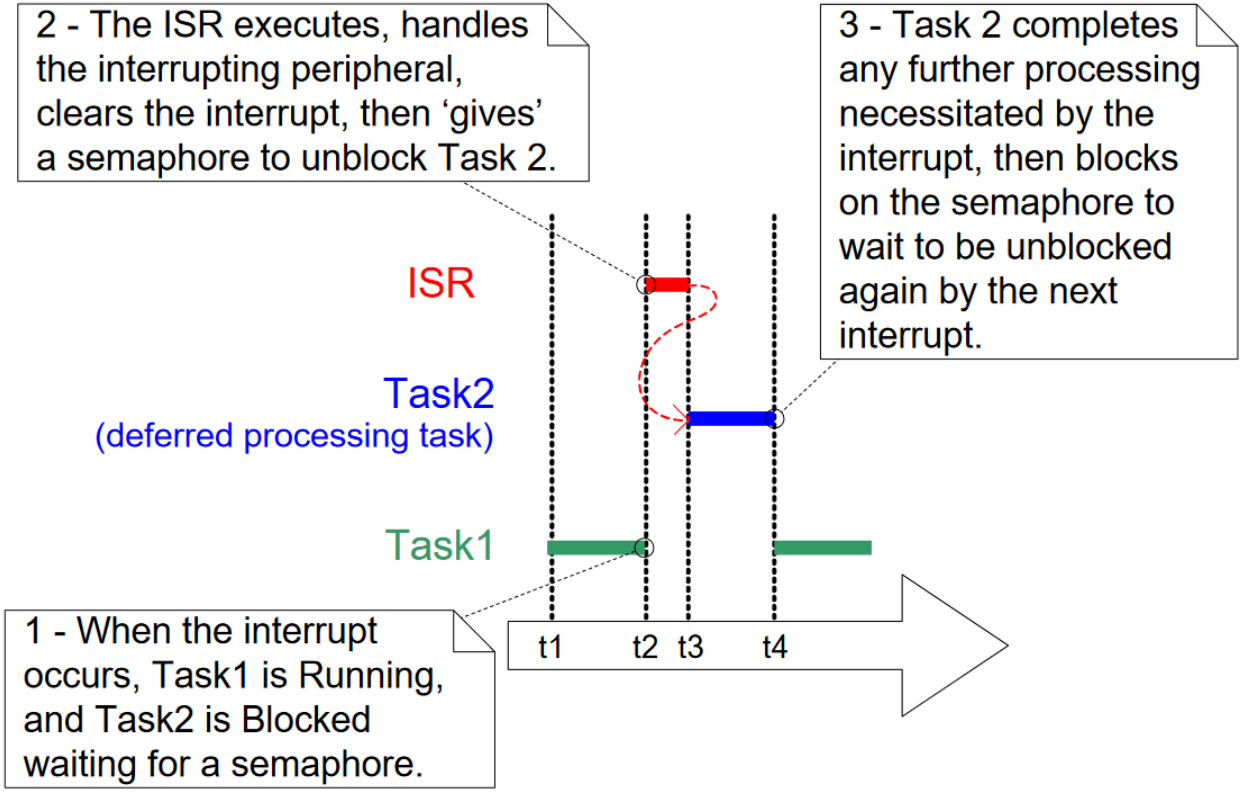
Двоичный семафор

ITMO UNIVERSITY

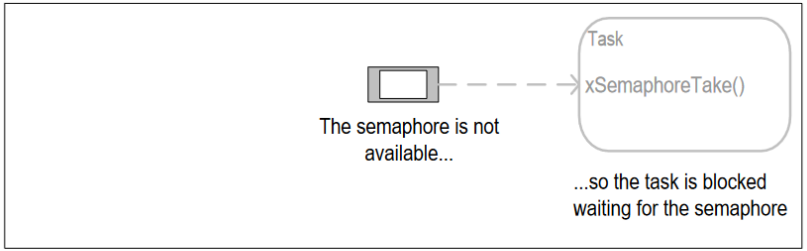
- Двоичные семафоры — это двоичные (0 или 1) флаги, которые можно установить как доступные или недоступные.

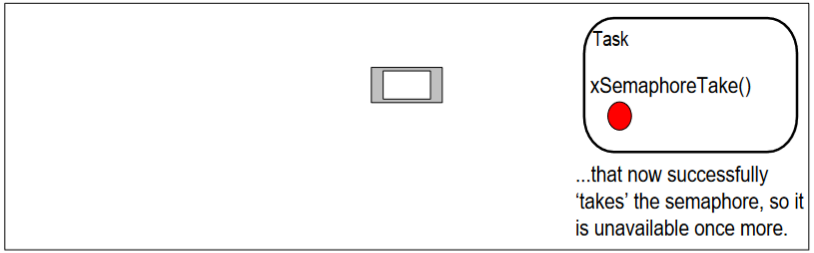
- Когда двоичный семафор используется для синхронизации задач, он изначально устанавливается равным 0 (пустым), поскольку он действует как событие, которого ожидают другие задачи. Другие задачи, которые должны выполняться в определенной последовательности, затем ждут (блокируются), пока двоичный семафор не станет равным 1 (пока не произойдет событие), чтобы взять семафор из исходной задачи и установить его обратно в 0.

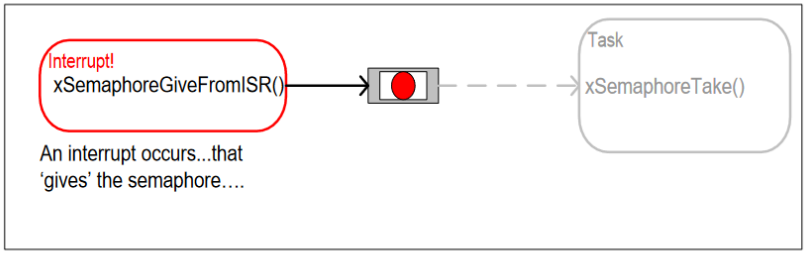
Отсрочка прерывания с помощью семафора

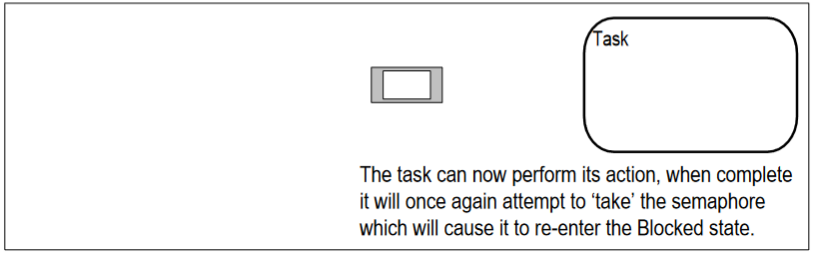


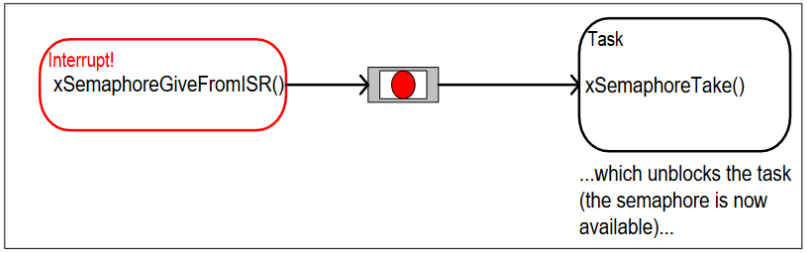
Отсрочка прерывания с помощью семафора



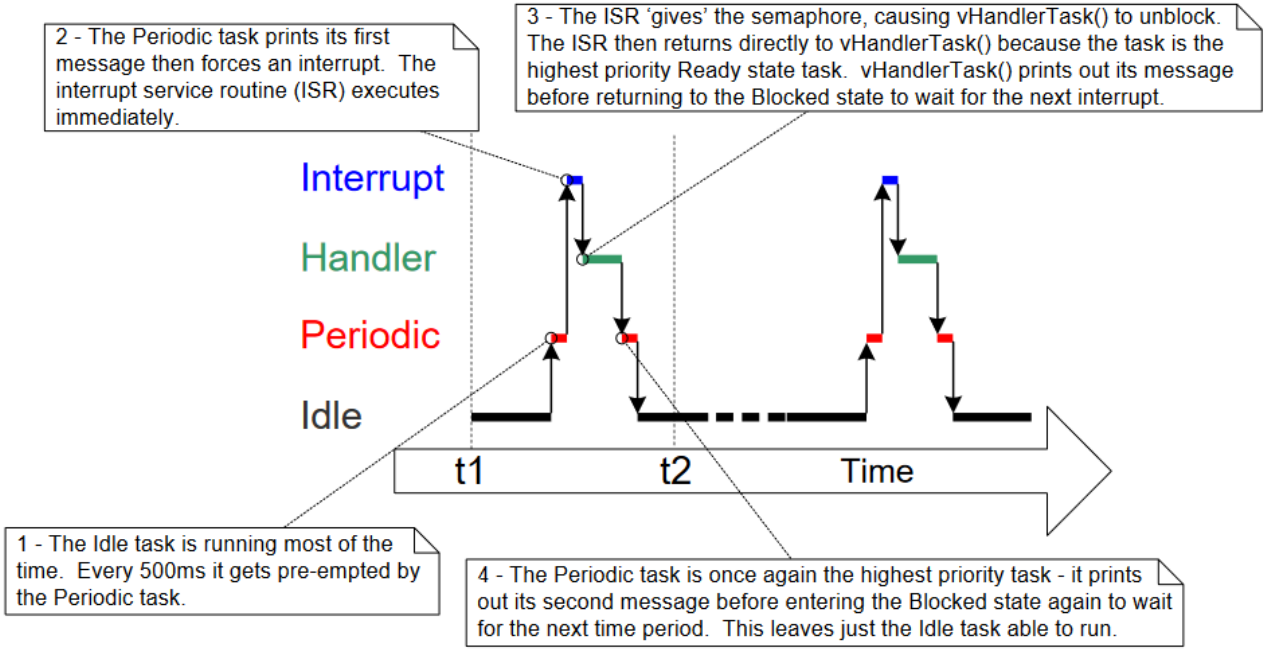








Двоичный семафор. Пример



Счетные семафоры

ITMO UNIVERSITY

- Так же, как двоичные семафоры можно рассматривать как очереди, имеющие длину один, семафоры-счетчики можно рассматривать как очереди, имеющие длину больше одного.

- Задачи не интересуются данными, которые хранятся в очереди, — только количеством элементов в очереди. configUSE_COUNTING_SEMAPHORES должен быть установлен в 1 в FreeRTOSConfig.h, чтобы счетные семафоры были доступны.

ITMO more than a UNIVERSITY

Счетные семафоры

ITMO UNIVERSITY

Семафоры подсчета обычно используются для двух вещей:

1.Подсчет событий

- В этом сценарии обработчик событий будет «давать» семафор каждый раз, когда происходит событие, заставляя значение счетчика семафора увеличиваться при каждом «передаче». Задача будет «брать» семафор каждый раз, когда она обрабатывает событие, заставляя значение счетчика семафора уменьшаться при каждом «взятии». Значение счетчика — это разница между количеством произошедших событий и количеством обработанных. Счетные семафоры, которые используются для подсчета событий, создаются с начальным значением счетчика, равным нулю.

Счетные семафоры

ITMO UNIVERSITY

Счетные семафоры обычно используются для двух вещей:

2. Управление ресурсами

- В этом сценарии значение счетчика указывает количество доступных ресурсов. Чтобы получить контроль над ресурсом, задача должна сначала получить семафор — уменьшая значение счетчика семафора. Когда значение счетчика достигает нуля, свободных ресурсов нет. Когда задача завершает работу с ресурсом, она «возвращает» семафор обратно — увеличивая значение счетчика семафора. Счетные семафоры, которые используются для управления ресурсами, создаются таким образом, чтобы их начальное значение счетчика равнялось количеству доступных ресурсов.

Общие ресурсы

ITMO UNIVERSITY

При доступе к периферийным устройствам рассмотрим следующий сценарий, в котором две задачи пытаются выполнить запись на жидкокристаллический дисплей (ЖК-дисплей).

1. Задача A выполняется и начинает записывать строку «Hello world» на ЖК-дисплей.
2. Задача A вытесняется задачей B после вывода только начала строки — «Hello w».
3. Задача B выводит «Abort, Retry, Fail?» на ЖК-дисплей перед переходом в заблокированное состояние.
4. Задача A продолжается с точки, в которой она была вытеснена, и завершает вывод оставшихся символов своей строки — «orld». Теперь на ЖК-дисплее отображается поврежденная строка «Hello wAbort, Retry, Fail?orld».

Взаимное исключение

ITMO UNIVERSITY

- Чтобы гарантировать постоянную согласованность данных, доступ к ресурсу, который совместно используется задачами или задачами и прерываниями, должен управляться с помощью техники «взаимного исключения». Цель состоит в том, чтобы гарантировать, что как только задача начинает обращаться к общему ресурсу, который не является реентерабельным и не потокобезопасным, та же задача имеет эксклюзивный доступ к ресурсу до тех пор, пока ресурс не будет возвращен в согласованное состояние.

- FreeRTOS предоставляет несколько функций, которые можно использовать для реализации взаимного исключения, но лучшим методом взаимного исключения является (по возможности, поскольку это часто непрактично) проектирование приложения таким образом, чтобы ресурсы не были общими, и доступ к каждому ресурсу осуществлялся только из одной задачи.

Критические разделы

ITMO UNIVERSITY

- Базовые критические разделы — это области кода, которые окружены вызовами макросов taskENTER_CRITICAL() и taskEXIT_CRITICAL() соответственно. Критические разделы также известны как критические регионы.

- Критические разделы, реализованные таким образом, являются очень грубым методом обеспечения взаимного исключения. Они работают путем отключения прерываний либо полностью, либо до приоритета прерывания, установленного configMAX_SYSCALL_INTERRUPT_PRIORITY. Упреждающие переключения контекста могут происходить только из прерывания, поэтому, пока прерывания остаются отключенными, задача, вызвавшая taskENTER_CRITICAL(), гарантированно останется в состоянии выполнения до выхода из критического раздела.

Мьютексы

ITMO UNIVERSITY

- Мьютекс — это особый тип двоичного семафора, который используется для управления доступом к ресурсу, который совместно используется двумя или более задачами. Слово MUTEX происходит от «MUTual EXclusion». configUSE_MUTEXES должен быть установлен в 1 в FreeRTOSConfig.h для доступности мьютексов.

- При использовании в сценарии взаимного исключения мьютекс можно рассматривать как токен, связанный с ресурсом, который совместно используется. Чтобы задача могла законно получить доступ к ресурсу, она должна сначала успешно «взять» токен (стать держателем токена). Когда держатель токена закончил работу с ресурсом, он должен «вернуть» токен. Только после возврата токена другая задача может успешно взять токен, а затем безопасно получить доступ к тому же общему ресурсу. Задаче не разрешается получать доступ к общему ресурсу, если она не владеет токеном.

Мьютексы против двоичных семафоров

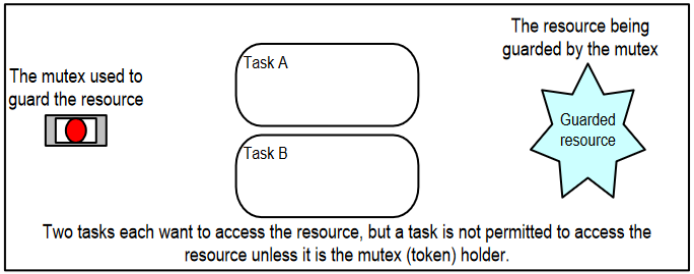
ITMO UNIVERSITY

- Несмотря на то, что мьютексы и двоичные семафоры имеют много общих характеристик, сценарии, в которых мьютекс используется для взаимного исключения, полностью отличаются от сценариев, в которых двоичный семафор используется для синхронизации.

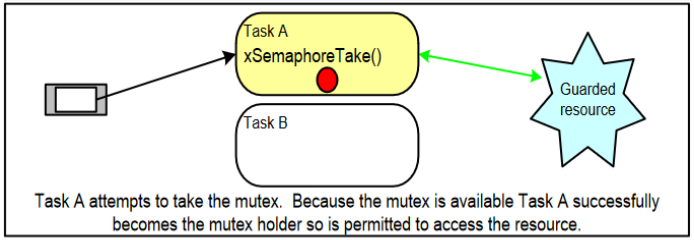
- Основное отличие заключается в том, что происходит с семафором/мьютексом после его получения:
  - Мьютекс имеет владельца и должен быть возвращен владельцем (поток/задача, которая захватила мьютекс)
  - Семафор не имеет владельца и может быть возвращен любой задачей

ITMO more than a UNIVERSITY

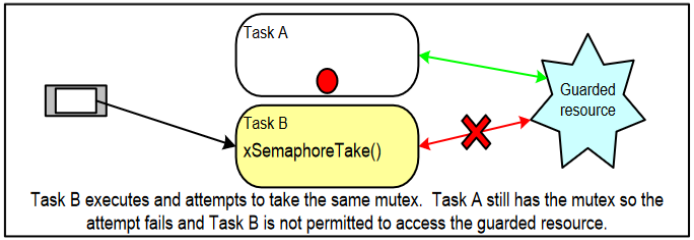
Использование мьютексов



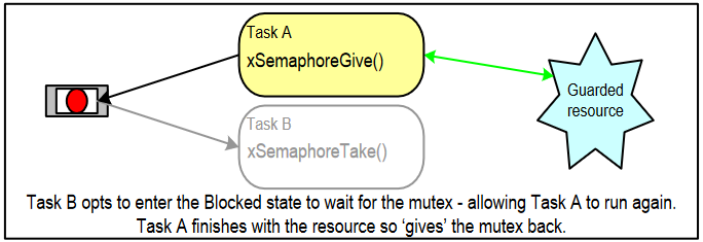
Two tasks each want to access the resource, but a task is not permitted to access the resource unless it is the mutex (token) holder.



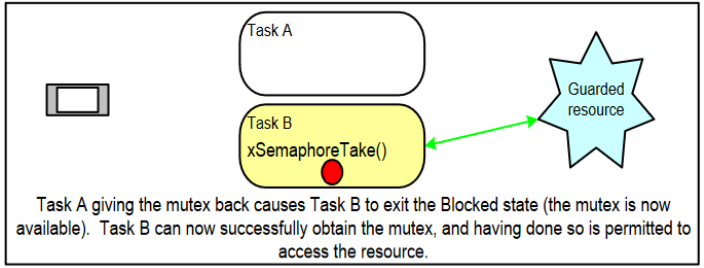
Task A attempts to take the mutex. Because the mutex is available Task A successfully becomes the mutex holder so is permitted to access the resource.



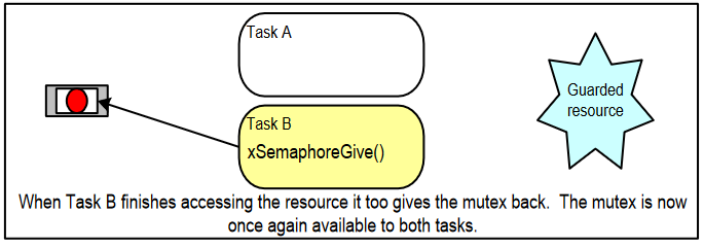
Task B executes and attempts to take the same mutex. Task A still has the mutex so the attempt fails and Task B is not permitted to access the guarded resource.



Task B opts to enter the Blocked state to wait for the mutex - allowing Task A to run again. Task A finishes with the resource so 'gives' the mutex back.



Task A giving the mutex back causes Task B to exit the Blocked state (the mutex is now available). Task B can now successfully obtain the mutex, and having done so is permitted to access the resource.



When Task B finishes accessing the resource it too gives the mutex back. The mutex is now once again available to both tasks.

Инверсия приоритетов

ITMO UNIVERSITY

- Одна из потенциальных ловушек использования мьютекса для обеспечения взаимного исключения называется «инверсией приоритета». Это ситуация, когда задача с более высоким приоритетом задерживается задачей с более низким приоритетом в ожидании мьютекса.

- Это нежелательное поведение будет еще больше усугубляться, если задача со средним приоритетом начнет выполняться, пока задача с высоким приоритетом будет ожидать семафор — результатом будет задача с высоким приоритетом, ожидающая задачу с низким приоритетом — без возможности выполнения задачи с низким приоритетом.

- Инверсия приоритета может быть серьезной проблемой, но в небольших встроенных системах ее часто можно избежать на этапе проектирования системы, продумав, как осуществляется доступ к ресурсам.

ITMO more than a UNIVERSITY

Взаимоблокировка (Deadlock)

ITMO UNIVERSITY

- «Взаимоблокировка» — еще одна потенциальная ловушка использования мьютексов для взаимного исключения. Взаимоблокировка иногда также известна под более драматичным названием «смертельные объятия». Взаимоблокировка происходит, когда две задачи не могут продолжить работу, потому что они обе ждут ресурс, удерживаемый другой.

- Также возможно, что задача вступит в взаимоблокировку сама с собой. Это произойдет, если задача попытается взять один и тот же мьютекс более одного раза, не вернув мьютекс предварительно. Такого типа взаимоблокировки можно избежать, используя **рекурсивный мьютекс** вместо стандартного мьютекса.

- Рекурсивный мьютекс может быть «взят» более одного раза одной и той же задачей и будет возвращен только после того, как один вызов «give» рекурсивного мьютекса будет выполнен для каждого предыдущего вызова «take» рекурсивного мьютекса.

Решение проблем

ITMO UNIVERSITY

- Как и в случае с инверсией приоритетов, лучший способ избежать взаимоблокировки — это учесть ее потенциал во время проектирования и спроектировать систему так, чтобы гарантировать невозможность возникновения взаимоблокировки.

- На практике взаимоблокировка не является большой проблемой в небольших встроенных системах, поскольку проектировщики системы могут хорошо понимать все приложение и, таким образом, могут определить и устранить области, где она может возникнуть.

ITMO more than a UNIVERSITY

FreeRTOS в STM32CubeIDE

ITMO UNIVERSITY

Подключение через .ioc файл:
Middleware and Software Packs -> FREERTOS -> Interface -> CMSIS_V1.

В открывшейся вкладке можно создавать задачи, очереди, семафоры, определять все необходимые параметры.

После генерации кода в вашем проекте появятся все необходимые библиотеки для FreeRTOS, а в файле freertos.c можно будет обнаружить все созданные ранее компоненты.

Пример: https://github.com/lmtspbru/SDK-1.1M/blob/master/SDK_FreeRTOS/

ITMOre than a UNIVERSITY

Пиксельная графика

ITMO UNIVERSITY

В рамках задания одним из вариантов работы с дисплеем будет использование пиксельной графики. Как вариант можно заранее создать примитивы, необходимые вам в приложении.

Пример: https://github.com/lmtspbru/SDK-1.1M/tree/master/SDK_OLED

SDK_OLED/Core/Src/fonts.c – примитивы для шрифтов

SDK_OLED/Core/Src/oled.c - char oled_WriteChar(char ch, FontDef Font, OLED_COLOR color) – работа с примитивами

Простейший путь: написать таблицу примитивов, необходимых вам и использовать их в дальнейшем.

ITMO more than a UNIVERSITY

Задание (группа до 3-х человек)

- Вариант 1. Реализовать игру «Змейка».
- Вариант 2. Реализовать игру «Тетрис».

Использование ОС FreeRTOS обязательно!

Пособие и примеры программ

- Учебное пособие: https://books.ifmo.ru/file/pdf/3031.pdf
- Примеры программ: https://github.com/lmtspbru/SDK-1.1M